

Звіт з лабораторної роботи №5

Тема: Автентифікація по Лемпорту

Мета роботи: дослідити принципи роботи систем одноразових паролів (ОТР) на основі хеш-ланцюгів. Спроекувати та реалізувати алгоритм автентифікації Лемпорта мовою Python, продемонструвати його стійкість до атак повторного відтворення та реалізувати механізми захисту від мережевої розсинхронізації і side-channel атак.

Виконав: студент групи ММШ-1 Красіков Антон

Теоретичні відомості

Алгоритм Лемпорта - це метод криптографічної автентифікації, який дозволяє клієнту безпечно підтверджувати свою особу серверу через незахищений канал зв'язку. В основі лежить використання криптографічної хеш-функції H та концепція хеш-ланцюгів.

Процес ініціалізації полягає у виборі секретного значення (seed) S та багаторазовому (рівно N разів) застосуванні до нього хеш-функції:

$$P_N = H^N(S)$$

Значення P_N передається на сервер і зберігається як еталон. Під час першої спроби входу клієнт надсилає P_{N-1} . Сервер перевіряє валідність пароля шляхом одноразового хешування отриманого значення:

$$H(P_{N-1}) == P_N$$

У разі успіху P_{N-1} стає новим еталоном на сервері для наступної ітерації $N-2$. Оскільки хеш-функція є необоротною, зловмисник, перехопивши поточний пароль P_i , не може обчислити наступний необхідний пароль P_{i-1} .

Спосіб вирішення

Для вирішення задачі було спроектовано клієнт-серверну архітектуру.

1. **Клієнтська частина** (*LamportClient*): відповідає за генерацію криптографічно стійкого секрету, побудову хеш-ланцюга заданої довжини та послідовну видачу одноразових паролів у зворотному порядку.

2. **Серверна частина (*LamportServer*)**: відповідає за зберігання поточного стану користувача (індекс та останній валідний хеш) та валідацію вхідних запитів.

При проектуванні архітектури було враховано дві критичні інженерні проблеми:

- **Вразливість до атак по часу**: стандартне порівняння рядків у пам'яті переривається на першому неспівпадінні байтів. Це дозволяє зловмиснику вимірювати час відповіді сервера та посимвольно підбирати хеш. Для вирішення цієї проблеми використано порівняння за константний час.
- **Проблема розсинхронізації**: якщо клієнт згенерував і відправив пароль, але пакет було втрачено, лічильник клієнта зменшиться, а сервера - ні. Щоб уникнути постійного блокування, на сервері імплементовано механізм *lookahead window*, який дозволяє пропускати кілька ітерацій хешування для відновлення синхронізації.

Деталі реалізації в коді

Реалізація виконана мовою Python з використанням стандартних бібліотек `hashlib` (для генерації SHA-256 хешів), `secrets` (для безпечної генерації ентропії) та `hmac`.

Для захисту від атак по часу замість оператора `==` використано функцію `hmac.compare_digest()`.

Реалізація клієнта:

```
def hash_data(data: bytes) -> bytes:
    """Calculates SHA-256 hash."""
    return hashlib.sha256(data).digest()

Windsurf: Refactor | Explain
class LamportClient:
    Windsurf: Refactor | Explain | Generate Docstring | X
    def __init__(self, user_id: str, seed_length: int = 32, chain_length: int = 100):
        self.user_id = user_id
        self.chain_length = chain_length
        self.current_index = chain_length
        self.logger = logging.getLogger("Client")

        # Generate a cryptographically secure random seed
        self._seed = secrets.token_bytes(seed_length)
        self._hash_chain = self._generate_chain()
        self.logger.info(f"Initialized client '{self.user_id}' with chain length {self.chain_length}.")

    Windsurf: Refactor | Explain | X
    def _generate_chain(self) -> list[bytes]:
        """Generates the full hash chain from P_0 to P_N."""
        chain = [self._seed]
        current = self._seed
        for _ in range(self.chain_length):
            current = hash_data(current)
            chain.append(current)
        return chain

    Windsurf: Refactor | Explain | X
    def get_registration_data(self) -> tuple[str, bytes, int]:
        """Returns data needed for server registration (User ID, P_N, N)."""
        self.logger.info("Exporting registration data (P_N) for secure channel transfer.")
        return self.user_id, self._hash_chain[self.chain_length], self.chain_length

    Windsurf: Refactor | Explain | X
    def generate_login_payload(self) -> bytes | None:
        """Generates the next one-time password (OTP) for login."""
        if self.current_index <= 0:
            self.logger.error("Hash chain exhausted! Re-registration required.")
            return None

        self.current_index -= 1
        otp = self._hash_chain[self.current_index]
        self.logger.info(f"Generated OTP for index {self.current_index}.")
        return otp
```

Реалізація сервера з підтримкою Lookahead Window:

```

class LamportServer:
    Windsurf: Refactor | Explain | Generate Docstring | ✕
    def __init__(self, lookahead_window: int = 5):
        # In a real app, this would be a database (e.g., PostgreSQL or Redis)
        self.db: dict[str, dict] = {}
        # Allows skipping a few hashes in case of network drops
        self.lookahead_window = lookahead_window
        self.logger = logging.getLogger("Server")

    Windsurf: Refactor | Explain | ✕
    def register_user(self, user_id: str, root_hash: bytes, chain_length: int) -> None:
        """Registers a user over a presumed secure channel."""
        self.db[user_id] = {
            "current_hash": root_hash,
            "current_index": chain_length
        }
        self.logger.info(f"Successfully registered user '{user_id}'.")

    Windsurf: Refactor | Explain | ✕
    def authenticate(self, user_id: str, provided_otp: bytes) -> bool:
        """Authenticates a user handling potential network desync."""
        if user_id not in self.db:
            self.logger.warning(f"Authentication failed: User '{user_id}' not found.")
            return False

        user_record = self.db[user_id]
        stored_hash = user_record["current_hash"]
        stored_index = user_record["current_index"]

        if stored_index <= 0:
            self.logger.warning(f"Authentication failed: User '{user_id}' hash chain exhausted.")
            return False

        # Lookahead verification: handle potential skipped indices due to network drops
        current_test_hash = provided_otp
        for offset in range(1, self.lookahead_window + 1):
            current_test_hash = hash_data(current_test_hash)

            # Using hmac.compare_digest to prevent timing attacks
            if hmac.compare_digest(current_test_hash, stored_hash):
                # Update state
                new_index = stored_index - offset
                self.db[user_id]["current_hash"] = provided_otp
                self.db[user_id]["current_index"] = new_index

                self.logger.info(
                    f"Auth SUCCESS for '{user_id}'. "
                    f"Index advanced from {stored_index} to {new_index} (Offset: {offset})."
                )
                return True

        self.logger.warning(f"Auth FAILED for '{user_id}': Invalid OTP or desync beyond lookahead window.")
        return False

```

Скрипт симуляції роботи системи (End-to-End):

```

def run_simulation():
    """Runs an end-to-end simulation of the Lamport scheme."""
    print("\nSTARTING LAMPORT AUTHENTICATION SIMULATION\n")

    server = LamportServer(lookahead_window=3)
    client = LamportClient(user_id="alice_dev", chain_length=10)

    print("\nPHASE 1: REGISTRATION (SECURE CHANNEL)")
    user_id, p_n, n = client.get_registration_data()
    server.register_user(user_id, p_n, n)

    print("\nPHASE 2: NORMAL LOGIN")
    otp_1 = client.generate_login_payload()
    server.authenticate(user_id, otp_1)

    print("\nPHASE 3: REPLAY ATTACK")
    server.logger.warning("Simulating network sniffer replaying the intercepted OTP...")
    server.authenticate(user_id, otp_1) # Sending the same OTP again

    print("\nPHASE 4: NETWORK DROP & DESYNC (LOOKAHEAD TEST)")
    client.logger.warning("Generating OTP, but simulating network drop (Server never receives it).")
    lost_otp = client.generate_login_payload()

    client.logger.info("Generating the NEXT OTP for a new login attempt.")
    otp_3 = client.generate_login_payload()

    server.logger.info("Server receives OTP_3. Expecting OTP_2. Testing lookahead recovery...")
    server.authenticate(user_id, otp_3)

    print("\nSIMULATION COMPLETE\n")

if __name__ == "__main__":
    run_simulation()

```

Результати виконання

Під час виконання симуляції було протестовано життєвий цикл системи: від реєстрації до відновлення після втрати мережових пакетів.

Вивід консолі (логи виконання):

```
STARTING LAMPORT AUTHENTICATION SIMULATION
```

```
00:09:59 | INFO      | Client      | Initialized client 'alice_dev' with
chain length 10.
```

```
PHASE 1: REGISTRATION (SECURE CHANNEL)
```

00:09:59 | INFO | Client | Exporting registration data (P_N) for secure channel transfer.

00:09:59 | INFO | Server | Successfully registered user 'alice_dev'.

PHASE 2: NORMAL LOGIN

00:09:59 | INFO | Client | Generated OTP for index 9.

00:09:59 | INFO | Server | Auth SUCCESS for 'alice_dev'. Index advanced from 10 to 9 (Offset: 1).

PHASE 3: REPLAY ATTACK

00:09:59 | WARNING | Server | Simulating network sniffer replaying the intercepted OTP...

00:09:59 | WARNING | Server | Auth FAILED for 'alice_dev': Invalid OTP or desync beyond lookahead window.

PHASE 4: NETWORK DROP & DESYNC (LOOKAHEAD TEST)

00:09:59 | WARNING | Client | Generating OTP, but simulating network drop (Server never receives it).

00:09:59 | INFO | Client | Generated OTP for index 8.

00:09:59 | INFO | Client | Generating the NEXT OTP for a new login attempt.

00:09:59 | INFO | Client | Generated OTP for index 7.

00:09:59 | INFO | Server | Server receives OTP_3. Expecting OTP_2. Testing lookahead recovery...

00:09:59 | INFO | Server | Auth SUCCESS for 'alice_dev'. Index advanced from 9 to 7 (Offset: 2).

SIMULATION COMPLETE

Аналіз результатів:

1. **Phase 1 & 2:** Система успішно зареєструвала користувача та провела першу автентифікацію. Індекс сервера оновився відповідно до очікувань.
2. **Phase 3 (Replay Attack):** Спроба повторно надіслати вже використаний пароль була успішно відхилена сервером, оскільки хеш від перехопленого P_{N-1} не дорівнює поточному еталону (яким вже став сам P_{N-1}).
3. **Phase 4 (Desync & Recovery):** Було зімітовано втрату одного ОТР-пароля. Наступний надісланий пароль знаходився "через один" крок від очікуваного сервером. Завдяки механізму *lookahead window*, сервер застосував хеш-функцію двічі, знайшов збіг з еталоном і успішно синхронізував індекси, поновивши роботу без ручного втручання.

Висновки

В ході виконання роботи було успішно реалізовано алгоритм автентифікації Лемпорта. Результати підтверджують, що алгоритм надійно захищає від пасивного перехоплення паролів (сніфінгу) та атак повторного відтворення (replay attacks), не вимагаючи передачі самого секрету мережею.

Однак, архітектурний аналіз виявив суттєві обмеження цієї системи для використання у сучасному production-середовищі:

1. **Statefulness:** Сервер змушений зберігати та оновлювати стан (поточний хеш та індекс) для кожного користувача. Це ускладнює горизонтальне масштабування в розподілених та мікросервісних архітектурах.

2. **Вичерпуваність:** Ланцюг має скінченну довжину N . Після використання всіх паролів потрібна повторна перереєстрація через захищений канал, що погіршує користувацький досвід.
3. **Вразливість до Man-in-the-Middle:** Хоча система захищає від пасивного перехоплення, активний зломисник може перехопити валідний запит, заблокувати його доставку на сервер і використати ОТР самостійно.

З огляду на ці недоліки, класичний алгоритм Лемпорта становить переважно академічну цінність як фундаментальна концепція хеш-ланцюгів. Для побудови сучасних систем одноразових паролів або багатофакторної автентифікації (MFA) доцільніше використовувати протокол TOTP, який є *stateless* і вирішує проблему вичерпуваності паролів.

Посилання на код (папка lab7):

<https://github.com/chesswondo/distributed-systems-labs>